

signup.js — Fundamentos

Explicación de la implementación de la pantalla de registro (GymBro App)

Es más largo que **login.js**, pero reutiliza casi las mismas ideas, con dos patrones nuevos: **estado como objeto único** y **función que genera funciones** (`setField`).

1. Las constantes de arriba (líneas 16-53)

```
const GENDERS = ['Femenino', 'Masculino'];
const GOALS = ['Salud', 'Bajar de peso', 'Fuerza', 'Tonificar', 'Otro'];
const EMPTY_FORM = { firstName: '', lastName: '', ... };
const REQUIRED_FIELDS = ['firstName', 'lastName', ..., 'goal'];
```

Están **fuera del componente**, a nivel de módulo. Eso es a propósito: son valores fijos que no dependen de ningún render, así que no tiene sentido recrearlos cada vez que `SignUpScreen` se re-renderiza (si estuvieran dentro del componente, se recrearían en cada render, desperdiciando memoria/tiempo sin ningún beneficio).

- **EMPTY_FORM** es la “plantilla” del formulario vacío — el estado inicial y también lo que usarías si quisieras resetear el form.
- **REQUIRED_FIELDS** es la lista de claves de `EMPTY_FORM` que son obligatorias (nota que `healthInsurance` y `contraindications` no están ahí — son opcionales).

2. Estado: un solo objeto en vez de un useState por campo (línea 84)

```
const [form, setForm] = useState(EMPTY_FORM);
```

A diferencia de `login.js` que tenía `useState` separado para email y password, acá con ~15 campos sería inmanejable tener 15 `useState`. En su lugar, **un solo estado que es un objeto**, y cada campo es una propiedad (`form.firstName`, `form.email`, etc.).

setField — función que devuelve una función (línea 88)

```
const setField = (field) => (value) => {
  setForm((prev) => ({ ...prev, [field]: value }));
  setError('');
};
```

Esto es lo más “raro” a primera vista, así que vale la pena desglosarlo:

- `setField` es una función que no actualiza nada directamente. Recibe el nombre de un campo (`'firstName'`) y devuelve otra función, que es la que efectivamente actualiza ese campo cuando se la llama con un valor.
- Se usa así: `onChangeText={setField('firstName')}`. Ahí `setField('firstName')` se ejecuta al renderizar, y da como resultado la función `(value) => {...}`. Esa función queda “enganchada” como `onChangeText`, y React Native se la va a llamar después con el texto que tipeó el usuario.
- Es el mismo patrón que en `login.js` hacías a mano con `onChangeText={text => { setEmail(text); setError(''); }}` — pero acá, en vez de escribir una función distinta a mano para cada uno de los ~15 inputs, generás la función con una fábrica (`setField`) parametrizada por el nombre del campo. Ahorra muchísima repetición.

- Adentro, `setForm((prev) => ({ ...prev, [field]: value }))`: `prev` es el form actual antes de esta actualización (usar la forma “callback” de `setState` es más seguro que `setForm({...form, ...})` cuando puede haber actualizaciones encoladas).
- `{ ...prev, [field]: value }` crea un objeto nuevo, copia todas las propiedades de `prev` (spread), y pisa solo la propiedad cuyo nombre es el valor de la variable `field` (esto se llama **computed property name**: `[field]` en vez de escribir el nombre literal). Si `field` es 'email', es como escribir `{ ...prev, email: value }`.
- Es inmutable: nunca se modifica `prev` directamente, siempre se genera un objeto nuevo — así React detecta el cambio correctamente.
- También limpia el error, igual que en login.

3. OptionChips — subcomponente para género y objetivo (líneas 56-77)

```
function OptionChips({ options, selected, onSelect }) {
  return (
    <View style={styles.optionsRow}>
      {options.map((option) => (
        <TouchableOpacity
          key={option}
          style={[styles.option, selected === option && styles.optionSelected]}
          onPress={() => onSelect(option)}
        >
          <Text style={[styles.optionText, selected === option && styles.optionTextSelected]}>
            {option}
          </Text>
        </TouchableOpacity>
      ))}
    </View>
  );
}
```

Es un componente **definido en el mismo archivo** (no exportado, uso privado de esta pantalla) que renderiza una fila de “pastillas” (chips) a partir de un array de opciones (GENDERS o GOALS).

- `options.map(...)` recorre el array y genera un `TouchableOpacity` por cada opción. `key={option}` es la key que React necesita para listas.
- `selected === option && styles.optionSelected`: si esta opción es la seleccionada, le agrega el estilo amarillo (patrón de estilos condicionales, típico en RN, ya visto en `login.js` con `isLandscape && styles.cardLandscape`).
- `onPress={() => onSelect(option)}`: al tocar, llama a la prop `onSelect` (que le pasa el padre) con el valor de esta opción.

Se usa dos veces:

```
<OptionChips options={GENDERS} selected={form.gender} onSelect={setField('gender')} />
<OptionChips options={GOALS} selected={form.goal} onSelect={setField('goal')} />
```

Fíjate que `onSelect` recibe `setField('gender')` — la misma función fábrica de antes. Cuando el usuario toca una pastilla, `OptionChips` llama `onSelect(option)`, que es `setField('gender')(option)`, que actualiza `form.gender`. Mismo mecanismo que los `CustomInput`, solo que la “fuente del valor” es un tap en vez de tipear.

4. handleRegister — la validación en cascada (líneas 95-138)

Es una serie de if que cortan con return apenas algo falla — patrón “guard clauses” (cláusulas de guarda), igual de legible que un único if gigante con &&, pero permite un mensaje de error distinto por cada regla:

- **Campos obligatorios:** `REQUIRED_FIELDS.some((field) => !form[field].trim())`. `.some()` devuelve true si al menos un campo requerido está vacío (`trim`). `form[field]` es acceso dinámico a la propiedad usando la variable `field` (mismo concepto de `computed access` que en `setField`).
- **Formato de email:** regex `/^\S+@\S+\.\S+$/` — “algo sin espacios, arroba, algo sin espacios, punto, algo sin espacios”. Es una validación básica de forma, no verifica que el dominio exista.
- **Formato de DNI:** regex `/^\d{7,8}$/` — exactamente 7 u 8 dígitos, sin puntos.
- **Longitud de contraseña:** mínimo 6 caracteres.
- **Coinciden las contraseñas:** `form.password !== form.confirmPassword`.

Números válidos (líneas 121-129)

```
const age = Number(form.age);
const weight = Number(form.weight.replace(',', ' '));
const height = Number(form.height.replace(',', ' '));
```

- ```
if (!Number.isInteger(age) || age <= 0 || !(weight > 0) || !(height > 0)) { ... }
```
- `.replace(',', ' ')` permite que el usuario escriba 70,5 (formato argentino) y lo convierte a 70.5 (formato que entiende `Number()`).
  - `Number.isInteger(age)` exige que la edad sea un entero (no 25.5).
  - `weight > 0 / height > 0`: si `Number(...)` no pudo convertir (por ejemplo el campo tiene letras), da NaN, y `NaN > 0` es false — por eso el `!(weight > 0)` detecta tanto negativos/cero como valores no numéricos, sin necesitar `isNaN()` explícito.

Recién si todas las validaciones pasan, se llama a `register(...)`:

```
const result = register({
 ...form,
 age: String(age),
 weight: String(weight),
 height: String(height),
});
if (!result.ok) setError(result.error ?? '');
```

- Manda una copia de `form` (`spread`) pero pisando `age/weight/height` con las versiones ya normalizadas (número validado, convertido de nuevo a string — así en `users.js` queda `age: '25'`, `weight: '70.5'` de forma prolija).
- `register` es la función del `AuthContext` (hace las validaciones de email/DNI duplicado y llama `addUser + setUser`).
- Igual que en `login.js`: si falla, muestra el error devuelto; si tiene éxito, no navega a mano — `register()` internamente hace `setUser(...)`, dispara el mismo mecanismo de `Context` → re-render de `AppNavigator` → cambia el stack de pantallas. Como los usuarios nuevos siempre tienen `isTrainer: false` (ver `addUser` en `users.js`), terminan directo en el stack de “usuario común” (`home_user`).

## 5. El resto del JSX

Es solo estructura visual: tres “secciones” (Datos Personales, Contacto y Salud, Entrenamiento) cada una envuelta en un `View` con estilo de tarjeta (`styles.section`), reutilizando `CustomInput` para cada

campo de texto y OptionChips para género/objetivo. No hay lógica nueva ahí — todo pasa por setField(...) y form.<campo> como ya vimos.

## En síntesis, comparado con login.js

|                            | login.js                       | signup.js                            |
|----------------------------|--------------------------------|--------------------------------------|
| <b>Estado</b>              | 2 useState sueltos             | 1 useState con objeto form           |
| <b>Actualizar un campo</b> | función inline por campo       | setField(field) fábrica reutilizable |
| <b>Validación</b>          | 1 regla (no vacíos) + backend  | 6 reglas en cascada + backend        |
| <b>Tras éxito</b>          | setUser desde login()          | setUser desde register()             |
| <b>Cambio de pantalla</b>  | automático vía Context (igual) | automático vía Context (igual)       |